

دانشگاه صنعتی شریف

گزارش بخش پیشرفته پروژه IMDb

معماری ارتباط با IMDb، تحلیل های API داخلی و پیاده سازی اتصال Flutter

دانشجو: مهرشاد ولیزاده

نوع گزارش: بخش پیشرفته پروژه

تاریخ: ۲۳ مرداد ۱۴۰۵

فهرست مطالب

۱	دامنه بخش پیشرفته و معماری کلی سامانه	۱
۱	۱.۱ مقدمه	۱
۱	۲.۱ اجزای اصلی	۱
۲	۳.۱ چرا Repository اهمیت دارد؟	۲
۲	۴.۱ جریان کلی داده	۲
۲	۵.۱ تفاوت بخش Client و Backend	۲
۲	۶.۱ جمع‌بندی	۲
۳	۲ آزمایش روش‌های اتصال و انتخاب Playwright	۳
۳	۱.۲ صورت مسئله	۳
۳	۲.۲ نتیجه آزمایش requests و urllib	۳
۳	۳.۲ چرا urllib به تنهایی کافی نبود؟	۳
۴	۴.۲ چرا requests نیز به تنهایی کافی نبود؟	۴
۴	۵.۲ آزمایش Playwright	۴
۴	۶.۲ دلیل انتخاب Playwright	۴
۵	۷.۲ نکته مهم درباره نقش curl_cffi	۵
۵	۸.۲ جمع‌بندی آزمایش	۵
۶	۳ ساخت Session با Playwright و استخراج Cookie	۶
۶	۱.۳ مقدمه	۶
۶	۲.۳ Header پایه	۶
۷	۳.۳ باز کردن Chromium	۷
۷	۴.۳ اجرای JavaScript و دریافت session-id	۷
۷	۵.۳ Cache کردن Session	۷
۸	۶.۳ HTML Scraping با Playwright	۸
۸	۷.۳ JSON-LD	۸
۸	۸.۳ جمع‌بندی	۸
۹	۴ های API IMDb مورد استفاده در پروژه	۹
۹	۱.۴ یک نکته مهم درباره واژه API	۹

۹	جست و جوی پیشنهادها	۲.۴
۹	Endpoint GraphQL	۳.۴
۱۰	Trending	۴.۴
۱۰	AdvancedTitleSearch	۵.۴
۱۰	FavoriteTitlesMetadata	۶.۴
۱۰	HERO_SUB_NAV_EPISODE	۷.۴
۱۰	EpisodeRatings_SeasonDetail	۸.۴
۱۱	جمع بندی ها Endpoint	۹.۴

۵ Query Persisted و ساختار GraphQL در IMDb ۱۲

۱۲	مقدمه	۱.۵
۱۲	ساختار درخواست GET	۲.۵
۱۳	ساخت درخواست POST	۳.۵
۱۳	Variables و Constraints	۴.۵
۱۳	نمونه Payload	۵.۵
۱۴	پردازش پاسخ	۶.۵
۱۴	مدیریت خطای GraphQL	۷.۵
۱۴	Cache	۸.۵
۱۴	جمع بندی	۹.۵

۶ جست و جوی پیشرفته و قابلیت های Discovery ۱۵

۱۵	جست و جوی عمومی	۱.۶
۱۵	تشخیص ID IMDb	۲.۶
۱۵	فیلتر نوع عنوان	۳.۶
۱۶	محدودیت امتیاز و تعداد رأی	۴.۶
۱۶	بازه زمانی	۵.۶
۱۶	Backend در Discover	۶.۶
۱۶	Sort	۷.۶
۱۶	جمع بندی	۸.۶

۷ Metadata، سریال ها، ها Episode و داده ویدئو ۱۷

۱۷	Metadata عنوان	۱.۷
۱۷	ترکیب IMDb و OMDb	۲.۷
۱۷	Overview Series	۳.۷
۱۸	های Episode یک فصل	۴.۷
۱۸	Scraping اطلاعات فیلم و سریال	۵.۷
۱۸	ویدئو	۶.۷
۱۹	جمع بندی	۷.۷

۲۰	۸	چگونگی اتصال مستقیم Flutter به IMDb
۲۰	۱.۸	نقطه اتصال در Flutter
۲۰	۲.۸	Endpoint مشترک
۲۰	۳.۸	های Flutter Header
۲۰	۴.۸	ارسال Request
۲۱	۵.۸	چرا Flutter توانست وصل شود؟
۲۱	۶.۸	دو مسیر متفاوت در پروژه
۲۱	۷.۸	مدیریت Timeout
۲۲	۸.۸	مدیریت پاسخ JSON
۲۲	۹.۸	OMDb با Fallback
۲۲	۱۰.۸	جمع‌بندی
۲۳	۹	Gateway بک‌اند و نقش آن در ارتباط با IMDb
۲۳	۱.۹	مقدمه
۲۳	۲.۹	View Search
۲۴	۳.۹	View Search Advanced
۲۴	۴.۹	View Episodes
۲۴	۵.۹	Views Discovery
۲۴	۶.۹	Scraper مستقل
۲۴	۷.۹	مزیت Gateway
۲۵	۸.۹	محدودیت Gateway
۲۵	۹.۹	جمع‌بندی
۲۶	۱۰	پایداری، امنیت، تست و محدودیت‌های راه‌حل
۲۶	۱.۱۰	Client در Cache
۲۶	۲.۱۰	Backend در Session Cache
۲۶	۳.۱۰	Timeout و خطا
۲۶	۴.۱۰	Fallback
۲۷	۵.۱۰	امنیت و اطلاعات حساس
۲۷	۶.۱۰	ریسک وابستگی به API داخلی IMDb
۲۷	۷.۱۰	وابستگی به Playwright
۲۷	۸.۱۰	تست‌های انجام‌شده
۲۷	۹.۱۰	تفاوت مشاهده تجربی و نتیجه قطعی
۲۸	۱۰.۱۰	بهبودهای پیشنهادی
۲۸	۱۱.۱۰	جمع‌بندی نهایی
۲۹	آ	پیوست: مرجع Endpoint‌ها و Operation‌های پروژه
۲۹	۱.آ	جدول Endpoint اصلی

۲۹	Client در Query Persisted Hash های	۲.آ
۳۰	مسیرهای Gateway محلی	۳.آ
۳۰	نکته پایانی	۴.آ

فصل ۱

دامنه بخش پیشرفته و معماری کلی سامانه

۱.۱ مقدمه

بخش پیشرفته پروژه با بخش عادی از این جهت تفاوت دارد که مسئله اصلی دیگر فقط ساخت رابط کاربری و نمایش داده نیست، بلکه باید مسیر واقعی رسیدن داده از سرویس‌های خارجی تا های Widget، Flutter نحوه عبور از محدودیت‌های ارتباط مستقیم با وبسایت IMDb، ساختار های API مورد استفاده و نقش لایه‌های میانی نیز بررسی شود. در نسخه پیاده‌سازی شده، دو مسیر برای دسترسی به داده‌های IMDb دیده می‌شود: مسیر مستقیم در Client فلاتر و مسیر واسط در Backend مبتنی بر Django. در Client، کلاس ImdbApiClient مستقیماً با caching.graphql.imdb.com و سرویس Suggestion ارتباط برقرار می‌کند. در کنار آن، OmdbApiClient یک منبع جانبی و اختیاری برای اطلاعات عنوان و فصل‌هاست. در Backend نیز های Django View درخواست‌هایی را به سرویس‌های IMDb ارسال کرده و در چند مورد داده خام را به JSON ساده‌تر برای مصرف Client تبدیل می‌کنند.

۲.۱ اجزای اصلی

معماری را می‌توان به پنج بخش مفهومی تقسیم کرد:

۱. **رابط کاربری: Flutter** مسئول دریافت ورودی کاربر، نمایش Loading و Error و نمایش داده مدل‌شده.
۲. **Repository:** مسئول انتخاب منبع داده، ترکیب چند پاسخ، اعمال Fallback و ارائه یک API ساده به UI.
۳. **API ها: Client** شامل ImdbApiClient و OmdbApiClient که جزئیات HTTP و JSON را پنهان می‌کنند.
۴. **Gateway: Backend** مجموعه های Django View که در صورت استفاده از مسیر Backend، به عنوان واسط بین Client و IMDb عمل می‌کنند.
۵. **IMDb:** منبع اصلی اطلاعات عنوان، جست‌وجو، Metadata Trending و Episode.

۳.۱ چرا Repository اهمیت دارد؟

اگر Widget مستقیماً، URL، Query Persisted Header و ساختار JSON را مدیریت می‌کرد، رابط کاربری به جزئیات فنی سرویس IMDb وابسته می‌شد. در پیاده‌سازی حاضر، UI متدهایی مانند `search`، `advancedSearch`، `titleDetailsBundle` و `seasonEpisodes` را از Repository می‌گیرد. بنابراین اگر منبع یک داده تغییر کند، لازم نیست های‌Widget متعدد بازنویسی شوند.

۴.۱ جریان کلی داده

برای نمونه، هنگام باز کردن جزئیات یک سریال، Repository ابتدا Metadata را از IMDb دریافت می‌کند، اطلاعات OMDb را نیز در صورت پیکربندی بررسی می‌کند و اگر عنوان قابلیت Episode داشته باشد، Overview سریال را نیز درخواست می‌کند. سپس این داده‌ها در TitleDetailsBundle کنار هم قرار می‌گیرند. نتیجه نهایی یک مدل قابل مصرف برای UI است، نه یک پاسخ خام GraphQL.

۵.۱ تفاوت بخش Backend و Client

وجود Backend به این معنی نیست که Flutter در همه موارد از آن استفاده می‌کند. در کد فعلی، Flutter دارای یک HttpClient داخلی Dart است و به صورت مستقیم به های‌IMDb Endpoint درخواست می‌فرستد. Backend نیز یک Gateway مستقل دارد که برای برخی قابلیت‌ها از Playwright و curl_cffi استفاده می‌کند. این دو مسیر از نظر معماری می‌توانند مکمل یکدیگر باشند و در گزارش باید از یکسان فرض کردن آن‌ها خودداری کرد.

۶.۱ جمع‌بندی

هسته بخش پیشرفته پروژه، جداسازی «منطق برنامه» از «نحوه دسترسی به IMDb» است. این جداسازی هم در Client با Repository و Client API و هم در Backend با های‌View تخصصی دیده می‌شود.

فصل ۲

آزمایش روش‌های اتصال و انتخاب Playwright

۱.۲ صورت مسئله

در مرحله بررسی امکان دسترسی برنامه‌نویسی به IMDb، چند روش متفاوت برای ارسال درخواست آزمایش شد. هدف این بود که بتوان بدون وابستگی به یک مرورگر کامل، صفحات یا های IMDb Endpoint را با یک HTTP Client معمولی دریافت کرد. در آزمایش‌های انجام‌شده روی لپ‌تاپ، استفاده از کتابخانه‌های رایج Python مانند requests و همچنین urllib به نتیجه مورد انتظار نرسید، در حالی که روش مبتنی بر Playwright موفق شد صفحه IMDb را در یک محیط Chromium بارگذاری کند.

۲.۲ نتیجه آزمایش requests و urllib

در این پروژه، شکست روش‌های requests و urllib یک مشاهده تجربی است و نباید آن را به صورت یک قانون عمومی درباره تمام های IMDb Endpoint تعبیر کرد. HTTP های Client معمولی می‌توانند درخواست HTTP را ارسال کنند، اما مسئله این بود که پاسخ دریافتی لزوماً معادل پاسخ یک مرورگر واقعی نبود. به بیان دیگر، صرفاً برقراری connection TCP/HTTPS به معنی عبور موفق از تمام کنترل‌های سمت وبسایت نیست. یکی از نشانه‌های مهم در پیاده‌سازی Backend این است که پس از بارگذاری صفحه IMDb، کد منتظر اجرای JavaScript می‌ماند و سپس Cookie با نام session-id را از Context مرورگر استخراج می‌کند. این رفتار نشان می‌دهد که بخشی از مسئله به Session و اجرای JavaScript در محیط مرورگر مربوط بوده است.

۳.۲ چرا urllib به تنهایی کافی نبود؟

urllib اساساً یک HTTP client است و صفحه را مانند مرورگر Render نمی‌کند. اگر سایت برای تولید یا تنظیم بخشی از وضعیت Session به اجرای JavaScript متکی باشد، دریافت HTML خام با urllib نمی‌تواند تمام مراحل مرورگر را بازسازی کند. حتی اگر های Header ظاهراً مشابه مرورگر اضافه شوند، باز هم ممکن است Cookie یا وضعیت Session مورد انتظار تولید نشوند.

۴.۲ چرا requests نیز به تنهایی کافی نبود؟

requests نسبت به urllib امکانات HTTP بیشتری دارد، اما همچنان Engine Browser نیست. در این پروژه نیز تجربه عملی نشان داد که ارسال یک GET ساده به IMDb به تنهایی راه حل قابل اتکایی برای عبور از مرحله Session نبود. بنابراین مسئله فقط User-Agent یا یک Header ساده نبود؛ فرآیند اجرای JavaScript و گرفتن Cookie در یک Context Browser اهمیت پیدا کرد.

۵.۲ آزمایش Playwright

Playwright یک مرورگر Chromium را در حالت Headless اجرا می‌کند. در Backend از Context جداگانه، User-Agent مشخص و اندازه صفحه ۱۹۲۰ در ۱۰۸۰ استفاده شده است. سپس صفحه اصلی IMDb باز می‌شود:

Listing 2.1: آزمایش‌های Playwright در Backend

```

1 with sync_playwright() as p:
2     browser = p.chromium.launch(
3         headless=True,
4         args=['--disable-blink-features=AutomationControlled']
5     )
6     context = browser.new_context(
7         user_agent=USER_AGENT,
8         viewport={'width': 1920, 'height': 1080}
9     )
10    page = context.new_page()
11    page.goto('https://www.imdb.com/',
12              wait_until='domcontentloaded', timeout=20000)
13    page.wait_for_timeout(5100)
14    cookies = context.cookies()
```

۶.۲ دلیل انتخاب Playwright

Playwright در این پروژه برای دو وظیفه مشخص مناسب بود. وظیفه اول ایجاد یک Session شبیه مرورگر و استخراج Cookie مورد نیاز بود. وظیفه دوم در BaseImdbScraper دریافت HTML کامل صفحه پس از Render شدن JavaScript و استخراج JSON-LD از DOM بود. در نتیجه Playwright هم برای «آماده‌سازی Session» و هم برای «Scraping صفحه Render شده» قابل استفاده شد.

۷.۲ نکته مهم درباره نقش `curl_cffi`

در نسخه Backend، بعد از اینکه Session و های Header مورد نیاز آماده شدند، برای بعضی درخواست‌های API از `curl_cffi` استفاده شده است. این کتابخانه جای Playwright را در مرحله اجرای JavaScript نمی‌گیرد؛ بلکه درخواست‌های HTTP بعدی را با امکان `impersonate="chrome110"` ارسال می‌کند. بنابراین معماری عملی به صورت «Browser» برای آماده‌سازی Client HTTP + Session برای «API» شکل گرفته است.

۸.۲ جمع‌بندی آزمایش

نتیجه تجربی پروژه را می‌توان چنین خلاصه کرد: Client HTTP ساده برای سناریوی آزمایش شده کافی نبود؛ Playwright توانست صفحه را در Chromium باز کند و Session را بسازد؛ سپس درخواست‌های API با های Header مناسب و در Backend با `curl_cffi` ادامه پیدا کردند. این نتیجه یکی از مهم‌ترین بخش‌های فنی مسیر توسعه بود.

فصل ۳

ساخت Session با Playwright و استخراج Cookie

۱.۳ مقدمه

یکی از قسمت‌های مهم Backend کلاس ImdbProxyBaseView است. این کلاس یک نقطه مشترک برای تمام‌هایی View است که لازم دارند با IMDb ارتباط برقرار کنند. هدف اصلی آن ساخت‌های Header مناسب و به‌دست آوردن session-id است.

۲.۳‌های Header پایه

در کد پروژه‌هایی Header مانند، Content-Type Referer، Accept-Language، Accept، User-Agent، و‌های Header مخصوص IMDb تنظیم شده‌اند. دو Header مهم عبارت‌اند از x-imdb-client-name و x-imdb-user-language. این مقادیر مطابق الگوی مورد استفاده در Client وب IMDb تنظیم شده‌اند.

Listing 3.1: IMDb با ارتباط برای استفاده‌های Header

```
1 headers = {  
2     'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) '  
3         'AppleWebKit/537.36 (KHTML, like Gecko) '  
4         'Chrome/110.0.0.0 Safari/537.36',  
5     'Accept': 'application/graphql+json, application/json',  
6     'Accept-Language': 'en-US,en;q=0.9',  
7     'Referer': 'https://www.imdb.com/',  
8     'Content-Type': 'application/json',  
9     'x-imdb-client-name': 'imdb-web-next-localized',  
10    'x-imdb-user-language': 'en-US',  
11    'x-imdb-user-country': 'US',  
12 }
```

۳.۳ باز کردن Chromium

Chromium یک Play --disable-blink-features=AutomationControlled گزینه باز می‌کند. در حالت Headless این گزینه به در پروژه استفاده شده است تا محیط مرورگر به شکل نزدیک‌تری به یک Browser عادی تنظیم شود. این گزینه به تنهایی تضمین‌کننده عبور از تمام سیستم‌های حفاظتی نیست و موفقیت نهایی در پروژه نتیجه اجرای واقعی صفحه، دریافت Cookie و سپس استفاده از Session بوده است.

۴.۳ اجرای JavaScript و دریافت session-id

بعد از `page.goto`، برنامه حدود پنج ثانیه منتظر می‌ماند. سپس های‌Cookie Context خوانده می‌شوند و Cookie با نام `session-id` استخراج می‌شود. اگر این Cookie وجود داشته باشد، مقدار آن به صورت Header `x-amzn-sessionid` به درخواست‌های بعدی اضافه می‌شود.

Listing 3.2: مرورگر Context از `session-id` استخراج

```

1 cookies = context.cookies()
2 session_id = None
3 for cookie in cookies:
4     if cookie['name'] == 'session-id':
5         session_id = cookie['value']
6         break
7
8 if session_id:
9     headers['x-amzn-sessionid'] = session_id
10 else:
11     raise ValueError('Failed to retrieve session-id')
```

۵.۳ Cache کردن Session

باز کردن Chromium برای هر درخواست بسیار پرهزینه است. به همین دلیل Session در سطح کلاس Cache می‌شود. در کد پروژه `_cache\_duration = 3600` است؛ یعنی های‌Header ساخته‌شده حداکثر یک ساعت دوباره استفاده می‌شوند و پس از آن Session جدید گرفته می‌شود. این تصمیم دو مزیت دارد: زمان پاسخ کاهش پیدا می‌کند و تعداد دفعات اجرای Chromium نیز کم می‌شود. در عوض، اگر Session زودتر از یک ساعت نامعتبر شود، درخواست‌ها ممکن است با Header قدیمی مواجه شوند؛ بنابراین این مقدار یک Trade-off مهندسی است.

۶.۳ HTML Scraping با Playwright

کلاس BaseImdbScraper علاوه بر Session، Proxy، یک مسیر جدا برای Scraping دارد. این کلاس URL را در Chromium باز می‌کند، پس از domcontentloaded چند ثانیه صبر می‌کند، سپس () page.content را می‌گیرد و Render HTML شده را با BeautifulSoup پردازش می‌کند.

۷.۳ JSON-LD

پس از دریافت HTML، اولین Script با نوع application/ld+json استخراج و به JSON تبدیل می‌شود. برای صفحه عنوان، اطلاعاتی مانند نام، تصویر، توضیح، ژانر، امتیاز تجمیعی و بازیگران از همین ساختار خوانده می‌شود. این روش با خواندن مستقیم متن ظاهری صفحه تفاوت دارد و به یک داده ساختاریافته متکی است.

۸.۳ جمع‌بندی

Playwright در پروژه صرفاً یک ابزار Scraping نیست. مهم‌ترین نقش آن در Gateway ایجاد یک Browser Context و آماده‌سازی Session است. پس از آن، درخواست‌های GraphQL و Suggestion می‌توانند با HTTP Client سبک‌تر انجام شوند.

فصل ۴

های API IMDb مورد استفاده در پروژه

۱.۴ یک نکته مهم درباره واژه API

هایی Endpoint که در این پروژه استفاده شده‌اند، بر اساس کد موجود، عمدتاً های Endpoint داخلی وبسایت IMDb هستند و نه یک API عمومی مستند شده که برای توسعه‌دهندگان شخص ثالث طراحی شده باشد. این تفاوت برای گزارش فنی مهم است. مسیرهایی مانند `caching.graphql.imdb.com` و های Operation مانند `AdvancedTitleSearch` و `EpisodeRatings\SeasonDetail` از ساختار داخلی سرویس وب IMDb استفاده می‌کنند.

بنابراین در این گزارش عبارت های API «IMDb» به معنای های Endpoint است که پروژه عملاً برای دریافت داده از زیرساخت وب IMDb فراخوانی کرده است، نه ادعای وجود یک قرارداد عمومی و پایدار برای این Endpointها.

۲.۴ API جست‌وجوی پیشنهادها

برای Search سریع از آدرس زیر استفاده شده است:

```
1 https://v3.sg.media-imdb.com/suggestion/x/{query}.json
```

پاسخ این Endpoint در فیلد d مجموعه‌ای از پیشنهادها را برمی‌گرداند. Client با متد `searchSuggestions` این داده را به `TitleSummary` تبدیل می‌کند. در Backend نیز `SearchView` همین سرویس را فراخوانی کرده و خروجی را تحت کلید `results` به Client برمی‌گرداند.

۳.۴ Endpoint GraphQL

بخش عمده داده‌های پیشرفته از Endpoint زیر دریافت می‌شود:

```
1 https://caching.graphql.imdb.com/
```

در این Endpoint، به جای چند URL مستقل، نام Operation و Variables تعیین می‌شوند. پروژه از Query Persisted نیز استفاده می‌کند؛ بنابراین به جای ارسال متن کامل Hash Query، آن در بخش extensions.persistedQuery.sha256Hash قرار می‌گیرد.

۴.۴ Trending

Client متد fetchTrending را دارد. Operation مورد استفاده Trending است و Variables شامل first و ورودی مربوط به پنجره زمانی و منبع ترافیک است. Hash استفاده‌شده در Client برابر است با:

```
419b4fc66817a78c3046e0cedef747033d5ac2711080338a59a366630f9742c1
```

نتیجه از مسیر data.topTrendingTitles.edges استخراج می‌شود.

۵.۴ AdvancedTitleSearch

این مهم‌ترین Endpoint جست‌وجوی پیشرفته پروژه است. Operation آن AdvancedTitleSearch است. Variables می‌توانند شامل عبارت جست‌وجو، نوع عنوان، ژانر، امتیاز، تاریخ انتشار، بازیگران، تعداد نتایج و ترتیب مرتب‌سازی باشند.

Hash مربوط به این Operation در کد Client و Backend یکسان است:

```
78932519bc74ceb6be628fe452c0e59a48bcf8ca91fc550dd5de43ab200acd52
```

۶.۴ FavoriteTitlesMetadata

با FavoriteTitlesMetadata Operation، اطلاعات Metadata برای مجموعه‌ای از IMDb ها ID دریافت می‌شود. Client شناسه‌ها را ابتدا پاک‌سازی و Unique می‌کند و سپس آن‌ها را در متغیر tconsts قرار می‌دهد.

۷.۴ HERO_SUB_NAV_EPISODE

برای Overview یک سریال از HERO_SUB_NAV_EPISODE Operation استفاده شده است. این درخواست برای تکمیل اطلاعات سریال، خصوصاً در مسیر نمایش جزئیات و قابلیت Episode کاربرد دارد.

۸.۴ EpisodeRatings_SeasonDetail

برای دریافت قسمت‌های یک فصل از EpisodeRatings_SeasonDetail Operation استفاده می‌شود. Variable شامل IMDb، Locale ID و شماره فصل هستند. نتیجه از مسیر تو در تو data.title.episodes.edges خوانده می‌شود.

۹.۴ جمع‌بندی ها Endpoint

در نتیجه، پروژه از یک خانواده Endpoint استفاده می‌کند: Suggestion برای جست‌وجوی سریع، GraphQL برای Search و Metadata Discovery، برای اطلاعات عنوان، Overview برای اطلاعات سریال و EpisodeRatings برای قسمت‌های یک فصل.

فصل ۵

Query Persisted و ساختار GraphQL در IMDb

۱.۵ مقدمه

در های API مورد استفاده پروژه، GraphQL به شکل سنتی «ارسال Query کامل در هر درخواست» استفاده نشده است. الگوی اصلی Query Persisted است. در این روش Operation و Variables به همراه یک Hash شناسه Query ارسال می شوند.

۲.۵ ساختار درخواست GET

برای هایی Operation مانند Metadata Trending، و Client Episodes از GET استفاده می کند. پارامترهای String Query شامل سه بخش اصلی هستند:

۱. operationName

۲. variables که به JSON تبدیل شده است

۳. extensions که شامل Query Persisted و Hash است

در کد Client، تابع `_getGraphQLPersisted` دقیقاً این ساختار را تولید می کند.

Listing 5.1: ساختار GET برای Persisted Query

```
1 final uri = _graphqlUri.replace(  
2   queryParameters: {  
3     'operationName': operationName,  
4     'variables': jsonEncode(variables),  
5     'extensions': jsonEncode(_persistedQuery(hash)),  
6   },
```

```

7 );
8 return _sendJson('GET', uri);

```

۳.۵ ساخت درخواست POST

برای Client Search، Advanced از POST استفاده می‌کند. Body شامل، Variables Operation و Extensions است:

Listing 5.2: ساخت POST برای AdvancedTitleSearch

```

1 body: jsonEncode({
2   'operationName': operationName,
3   'variables': variables,
4   'extensions': _persistedQuery(hash),
5 })

```

۴.۵ Constraints و Variables

مزیت اصلی Variables این است که یک Operation می‌تواند با پارامترهای مختلف استفاده شود. برای مثال Search Advanced در Client می‌تواند فقط عبارت جست‌وجو را بفرستد یا همزمان محدودیت نوع عنوان، تاریخ، امتیاز و تعداد رأی را نیز اضافه کند.

برای نوع عنوان از titleTypeConstraint استفاده شده است. برای امتیاز از userRatingsConstraint و برای تاریخ از releaseDateConstraint استفاده می‌شود. برای Movies Rated Top نیز محدودیت rankedTitleListConstraint به Variables افزوده می‌شود.

۵.۵ نمونه Payload

نمونه ساده‌شده زیر منطق Payload پروژه را نشان می‌دهد:

Listing 5.3: نمونه Payload منطقی AdvancedTitleSearch

```

1 {
2   "operationName": "AdvancedTitleSearch",
3   "variables": {
4     "locale": "en-US",
5     "first": 20,
6     "sortBy": "POPULARITY",
7     "sortOrder": "ASC",
8     "titleTextConstraint": {

```

```

9      "searchTerm": "Breaking Bad"
10    },
11    "titleTypeConstraint": {
12      "anyTitleTypeIds": ["tvSeries"]
13    }
14  },
15  "extensions": {
16    "persistedQuery": {
17      "version": 1,
18      "sha256Hash": "...
19    }
20  }
21 }

```

۶.۵ پردازش پاسخ

بعد از دریافت پاسخ، Client به صورت مستقیم کل JSON را به UI نمی‌دهد. برای مثال در Search Advanced مسیر `data.advancedTitleSearch.edges` خوانده می‌شود. هر Edge سپس به `TitleSummary` تبدیل می‌شود. این کار باعث می‌شود تغییرات جزئی در ساختار GraphQL در یک لایه محدود بماند.

۷.۵ مدیریت خطای GraphQL

کد `_sendJson` علاوه بر `Code`، `Status` وجود فیلد `errors` در JSON را نیز بررسی می‌کند. بنابراین HTTP ۲۰۰ به تنهایی موفقیت عملیات فرض نمی‌شود. اگر GraphQL خطا برگرداند، یک `ImdbApiException` با پیام خطا ساخته می‌شود.

۸.۵ Cache

در Client، نتیجه JSON هر درخواست با یک Key Cache شامل `URI Method` و `Body` نگهداری می‌شود. TTL این Cache پانزده دقیقه است. این Cache از ارسال درخواست تکراری برای یک Query یکسان جلوگیری می‌کند.

۹.۵ جمع‌بندی

Query Persisted باعث شده Client بتواند عملیات‌های مشخص IMDb را با های‌هاش مربوط به آن‌ها فراخوانی کند. در نتیجه بخش مهمی از پیچیدگی ارتباط با IMDb در `ImdbApiClient` متمرکز شده است.

فصل ۶

جستوجوی پیشرفته و قابلیت‌های Discovery

۱.۶ جستوجوی عمومی

دو مسیر متفاوت در Client وجود دارد. `searchSuggestions` برای پیشنهادهای سریع و `autocomplete` مناسب است و `advancedTitleSearch` برای جستوجوی ساختاریافته استفاده می‌شود. `Repository` بر اساس نیاز UI این دورا از هم جدا می‌کند.

۲.۶ تشخیص ID IMDb

یکی از قابلیت‌های جالب `Repository` این است که اگر کاربر عبارتی شامل الگوی `tt\d+` وارد کند، برنامه آن را `ID IMDb` در نظر می‌گیرد. در این حالت به جای `Search` متنی، `Metadata` مستقیماً برای آن `ID` درخواست می‌شود.

Listing 6.1: تشخیص ID IMDb در `Repository`

```
1 final match = RegExp(
2     r'tt\d+',
3     caseSensitive: false,
4 ).firstMatch(value.trim());
```

این رفتار باعث می‌شود ورودی‌هایی مانند `tt1234567` به عنوان شناسه تفسیر شوند و نیاز به `Search` متنی کاهش پیدا کند.

۳.۶ فیلتر نوع عنوان

`Search Advanced` می‌تواند نوع اثر را با شناسه‌هایی مانند `movie`، `tvSeries` و `tvMiniSeries` محدود کند. `Repository` برای `Search` فیلم فقط `movie` و برای `Search` سریال دو نوع سریال را می‌فرستد.

۴.۶ محدودیت امتیاز و تعداد رأی

در Client امکان تعیین minimumRating و minimumVotes وجود دارد. این دو مقدار در userRatingsConstraint قرار می‌گیرند. همچنین گزینه Movies Rated Top یک List Ranked را هدف قرار می‌دهد و بازه رتبه را تا ۲۵۰ اثر اول محدود می‌کند.

۵.۶ بازه زمانی

برای جست‌وجوی آثار جدید، Repository می‌تواند تاریخ شروع و پایان را به releaseDateConstraint اضافه کند. Backend نیز یک View با نام NewReleasesView دارد که تاریخ ۳۰ روز گذشته تا امروز را محاسبه و به Search Advanced تزریق می‌کند.

۶.۶ Backend در Discover

Backend پنج View مشخص برای Discovery دارد:

۱. Movies Popular

۲. Shows TV Popular

۳. Releases New

۴. Rated Top

۵. Recommended

چهار مورد اول با تغییر پارامترهای Search Advanced ساخته می‌شوند. برای Recommended، چون در این پروژه یک پروفایل سلیقه کاربر برای موتور پیشنهاددهی وجود ندارد، Backend یک ژانر را به صورت تصادفی از فهرستی شامل Crime، Comedy Thriller، Mystery، Sci-Fi، Action انتخاب می‌کند و Search را بر اساس آن اجرا می‌کند.

۷.۶ Sort

پارامترهای sortBy و sortOrder به API منتقل می‌شوند. مقادیر پیش‌فرض Backend برای بسیاری از درخواست‌ها POPULARITY و ASC است، در حالی که Rated Top با USER_RATING و DESC تنظیم می‌شود.

۸.۶ جمع‌بندی

جست‌وجوی پیشرفته پروژه یک Box Search ساده نیست؛ بلکه یک لایه Builder Query دارد که بسته به نوع درخواست، Variables مناسب GraphQL را می‌سازد. این قابلیت یکی از مهم‌ترین نقاط بخش پیشرفته است.

فصل ۷

Metadata، سریال‌ها، ها Episode و داده ویدئو

۱.۷ Metadata عنوان

FavoriteTitlesMetadata Operation برای دریافت اطلاعات مجموعه‌ای از IMDb ها ID استفاده می‌شود. Client ابتدا ID ها را Trim و Unique می‌کند و سپس آن‌ها را در متغیر tconsts قرار می‌دهد. نتیجه از data.titles خوانده می‌شود.

مدل TitleDetails داده‌هایی مانند عنوان، تصویر، توضیح، ژانر، امتیاز، تعداد رأی، سال، مدت و قابلیت Episode را برای استفاده UI نگه می‌دارد.

۲.۷ ترکیب IMDb و OMDb

برای صفحه جزئیات، Repository متد titleDetailsBundle را اجرا می‌کند. در این متد Metadata IMDb و اطلاعات OMDb به صورت مستقل دریافت می‌شوند. سپس در صورت سریال بودن اثر، Series Overview نیز درخواست می‌شود.

OMDb در این پروژه منبع اصلی IMDb محسوب نمی‌شود. آن یک سرویس جانبی با Endpoint مستقل www.omdbapi.com است و با کلید API فعال می‌شود. وجود آن برای Fallback اهمیت دارد؛ بنابراین باید در گزارش از های API داخلی IMDb و OMDb به عنوان دو منبع متفاوت یاد شود.

۳.۷ Overview Series

برای سریال‌ها HERO_SUB_NAV_EPISODE Operation استفاده می‌شود. Repository ابتدا تشخیص می‌دهد عنوان قابلیت Episode دارد و سپس این درخواست را انجام می‌دهد. در صورت شکست، خطا در Bundle ثبت می‌شود تا UI بتواند داده موجود را بدون الزام به موفقیت تمام منابع نمایش دهد.

۴.۷ های Episode یک فصل

EpisodeRatings_SeasonDetail Operation با متغیرهای id، locale و seasons اجرا می‌شود. Backend نیز همین Operation را در EpisodesView استفاده می‌کند. در Backend، هر Node به ساختاری ساده‌تر تبدیل می‌شود:

Listing 7.1: ساده JSON به Episode خام IMDb تبدیل

```

1 episode_data = {
2     'id': node.get('id'),
3     'title': node.get('titleText', {}).get('text'),
4     'season_number': ...,
5     'episode_number': ...,
6     'release_date': ...,
7     'plot': ...,
8     'image_url': ...,
9 }
```

۵.۷ Scraping اطلاعات فیلم و سریال

Backend در کنار GraphQL، یک Scraper عمومی نیز دارد. BaseImdbScraper صفحه عنوان را با Playwright باز می‌کند و Render HTML شده را به BeautifulSoup می‌دهد. سپس JSON-LD استخراج می‌شود.

ای سریال، سال شروع و پایان از لینک /releaseinfo/ و کشور از عنصر دارای "title-details-origin" data-testid خوانده می‌شود. برای فیلم، سال از datePublished و مدت از قالب ISO مانند PT1H54M استخراج و به قالب خوانا تر تبدیل می‌شود.

۶.۷ ویدئو

Backend یک View با مسیر /api/video/<video_id>/ دارد. کلاس VideoScraper صفحه ویدئو را با Playwright باز می‌کند و Script با شناسه __NEXT_DATA__ را می‌خواند. سپس از ساختار videoPlaybackData اطلاعاتی مانند ID، عنوان، Runtime Thumbnail و Playback ها URL استخراج می‌شود. این بخش نیز نشان می‌دهد که برای بعضی داده‌ها لازم بوده به ساختار Render شده و داده‌های داخلی صفحه تکیه شود، نه فقط یک HTML خام.

۷.۷ جمع‌بندی

داده‌های عنوان و سریال در پروژه از چند مسیر جمع می‌شوند: GraphQL داخلی IMDb برای داده‌های ساختاریافته، HTML/JSON-LD برای Scraping و OMDb به عنوان منبع جانبی. Repository این منابع را به یک مدل قابل مصرف برای Flutter تبدیل می‌کند.

فصل ۸

چگونگی اتصال مستقیم Flutter به IMDb

۱.۸ نقطه اتصال در Flutter

یکی از نکات کلیدی پروژه این است که Flutter برای ارتباط مستقیم با IMDb به Package خارجی HTTP وابسته نیست. در ImdbApiClient از dart:io و کلاس داخلی HttpClient استفاده شده است. بنابراین Flutter می‌تواند Request HTTPS را مستقیماً از Client ارسال کند.

۲.۸ Endpoint مشترک

در کد Flutter یک URI ثابت برای GraphQL تعریف شده است:

```
1 static final Uri _graphqlUri =  
2   Uri.https('caching.graphql.imdb.com', '/');
```

این URI سپس برای عملیات‌های مختلف با Parameters Query یا Body متفاوت استفاده می‌شود.

۳.۸ Flutter Header های

Flutter Client نیز هایی Header مشابه Header مورد نیاز Backend دارد. از جمله، Accept- Accept، User-Agent Referer، Content-Type، Language، defaultHeaders در این هاHeader x-imdb-* و هایHeader در Request HTTP قرار می‌گیرند.

۴.۸ ارسال Request

متد _sendJson اصلی ارسال Request را بر عهده دارد. ابتدا Cache بررسی می‌شود. سپس Request با openUrl ساخته می‌شود، هاHeader اضافه می‌شوند، Body در صورت وجود نوشته می‌شود و پاسخ خوانده می‌شود.

Listing 8.1: درخواست HTTP در Flutter ارسال

```

1 final request = await _httpClient
2   .openUrl(method, uri)
3   .timeout(timeout);
4
5 defaultHeaders.forEach(request.headers.set);
6 if (body != null) {
7   request.add(utf8.encode(body));
8 }
9 final response = await request.close().timeout(timeout);

```

۵.۸ چرا Flutter توانست وصل شود؟

تفاوت مهم اینجا با آزمایش اولیه Python این است که «زبان» یا Flutter «بودن» به خودی خود باعث عبور از محدودیت‌های IMDb نمی‌شود. در Client فعلی، درخواست مستقیم با های Header مورد انتظار و های Endpoint مشخص GraphQL انجام می‌شود. در مقابل، آزمایش Python با requests و urllib در سناریوی اولیه برای دسترسی مورد نظر موفق نبود.

بنابراین نباید نتیجه گرفت که Flutter ذاتاً از محدودیت IMDb عبور می‌کند. توضیح دقیق‌تر این است که Flutter Client مستقیماً های GraphQL Endpoint و Suggestion را با ساختار Request مناسب فراخوانی می‌کند و در مسیر Lookup ID و Episode نیز های Fallback مشخص دارد. در این کد، مکانیزم Playwright برای Flutter Client وجود ندارد؛ Playwright مربوط به Gateway و Backend Scraper است.

۶.۸ دو مسیر متفاوت در پروژه

برای جلوگیری از اشتباه معماری، دو جریان زیر باید جدا در نظر گرفته شوند:

۱. **Flutter Direct:** Flutter با HttpClient به IMDb/OMDb متصل می‌شود.

۲. **Backend Gateway: Django:** Session Playwright می‌سازد و سپس با curl_cffi یا Scraper به IMDb متصل می‌شود.

در نتیجه اگر از Client مستقیم استفاده شود، درخواست از Flutter به IMDb می‌رود و Backend لزوماً در مسیر آن درخواست نیست. اگر قابلیت از Gateway استفاده کند، مسیر از Django عبور می‌کند.

۷.۸ مدیریت Timeout

Timeout پیش‌فرض ImdbApiClient دوازده ثانیه است. OMDb ده ثانیه Timeout دارد. این محدودیت باعث می‌شود UI در صورت قطع شبکه برای مدت نامحدود در حالت Loading باقی نماند.

۸.۸ مدیریت پاسخ JSON

پاسخ ابتدا به متن UTF-۸ تبدیل می‌شود و سپس با `jsonDecode` خوانده می‌شود. اگر JSON یک Object نباشد، Exception مناسب ساخته می‌شود. همچنین وجود فیلد GraphQL به نام `errors` بررسی می‌شود.

۹.۸ OMDb با Fallback

در ID Lookup، اگر Metadata داخلی IMDb شکست بخورد، Repository تلاش می‌کند اطلاعات عنوان را از OMDb بگیرد. برای Episode نیز ابتدا IMDb و سپس OMDb بررسی می‌شود. بنابراین اتصال Flutter فقط یک درخواست واحد نیست و یک زنجیره مقاوم‌تر از منابع دارد.

۱۰.۸ جمع‌بندی

Flutter به کمک `HttpClient`، های Header مشخص، `Persisted`، `JSON Query`، `Cache decoding` و `handling Exception` مستقیماً با های `IMDb Endpoint` کار می‌کند. `Playwright` بخشی از راه‌حل Backend است و نباید با مسیر مستقیم Flutter اشتباه گرفته شود.

فصل ۹

Gateway بک‌اند و نقش آن در ارتباط با IMDb

۱.۹ مقدمه

Backend پروژه با Django و Django REST Framework ساخته شده و چند Endpoint محلی را در اختیار Client قرار می‌دهد. این Gateway می‌تواند داده IMDb را دریافت، ساده‌سازی و به ساختار مورد انتظار برنامه تبدیل کند.

۲.۹ View Search

مسیر `/api/search/` پارامتر `q` را می‌گیرد. ابتدا `ImdbProxyBaseView.get\_imdb\_headers()` اجرا می‌شود و سپس `IMDb Suggestion Endpoint` با `curl\_cffi` فراخوانی می‌شود. پاسخ `results` به `d` تبدیل می‌شود.

Listing 9.1: Search در Gateway

```
1 safe_query = urllib.parse.quote(query)
2 url = f'https://v3.sg.media-imdb.com/suggestion/x/{safe_query}.json'
3 request_headers = self.get_imdb_headers()
4 response = cffi_requests.get(
5     url,
6     headers=request_headers,
7     params={'includeVideos': '1'},
8     impersonate='chrome110',
9     timeout=10,
10 )
```

۳.۹ View Search Advanced

مسیر `/api/search/advanced/` پارامترهای ساده HTTP را به GraphQL Variables تبدیل می‌کند. برای مثال type به `titleTypeConstraint`، ژانر به `genreConstraint` و حداقل امتیاز به `userRatingsConstraint` تبدیل می‌شود.

۴.۹ View Episodes

مسیر `/api/episodes/` شناسه عنوان و شماره فصل را می‌گیرد. سپس `EpisodeRatings\_SeasonDetail Operation` را با GET به Endpoint GraphQL ارسال می‌کند. پس از دریافت پاسخ، Backend ساختار پیچیده `Edge/N-` را به JSON ساده Episode تبدیل می‌کند.

۵.۹ Views Discovery

های `Discovery View` از `Search Advanced` به عنوان کلاس پایه استفاده می‌کنند. این طراحی از تکرار `Builder Query` جلوگیری می‌کند. برای `Movies Popular` فقط نوع عنوان روی `Movie` قرار می‌گیرد؛ برای `Shows TV Popular` نوع عنوان روی `Series TV` و `Series Mini TV` تنظیم می‌شود؛ برای `Rated Top` حداقل امتیاز ۸ و مرتب‌سازی بر اساس `Rating User` اعمال می‌شود.

۶.۹ Scraper مستقل

`Gateway` علاوه بر `API`، یک `Proxy` Scraper مستقل دارد. این Scraper در مواقعی که داده در `HTML` صفحه یا `JSON-LD` قابل دسترسی است، از `Playwright` و `BeautifulSoup` استفاده می‌کند. بنابراین Backend دو الگوی متفاوت دارد:

۱. **Proxy: API** دریافت Session با `Playwright` و سپس ارسال `GraphQL/HTTP` با `curl_cffi`.

۲. **Scraping: Browser** باز کردن URL در `Chromium`، دریافت `HTML Render` شده و استخراج داده.

۷.۹ مزیت Gateway

بخشی از پیچیدگی IMDb را از Client حذف کند. برای مثال Client می‌تواند فقط `&season=1` را به `/api/episodes/?id=tt...` اضافه کند. این کار باعث می‌شود Backend جزئیات `Hash Operation` و مسیر `JSON` را مدیریت نکند.

۸.۹ محدودیت Gateway

این Gateway نیز به ساختارهای داخلی IMDb وابسته است. اگر، Operation، Header Hash، مورد نیاز یا ساختار JSON تغییر کند، View ممکن است نیاز به اصلاح داشته باشد. همچنین اجرای Chromium روی سرور نسبت به یک Request HTTP ساده منابع بیشتری مصرف می‌کند.

۹.۹ جمع‌بندی

Backend یک لایه Abstraction واقعی ایجاد می‌کند، اما هدف آن در این پروژه صرفاً «عبور دادن همه درخواست‌ها» نیست. در بخش‌هایی داده را تبدیل و ساده می‌کند، در بخش‌هایی Session می‌سازد و در بخش‌هایی HTML را Scrape می‌کند.

فصل ۱۰

پایداری، امنیت، تست و محدودیت‌های راه‌حل

۱.۱۰ Client در Cache

Cache پانزده دقیقه‌ای در `ImdbApiClient` و `OmdbApiClient` باعث می‌شود درخواست‌های تکراری به یک `URI` و `Body` یکسان دوباره ارسال نشوند. این مسئله مخصوصاً برای صفحه‌هایی که چند `Widget` یک داده مشترک را درخواست می‌کنند اهمیت دارد.

۲.۱۰ Backend در Session Cache

`IMDb Session Backend` را تا یک ساعت `Cache` می‌کند. این تصمیم هزینه اجرای `Chromium` را کاهش می‌دهد. با این حال، `Session` یک داده موقتی است و نباید به عنوان یک `Credential` دائمی تلقی شود.

۳.۱۰ Timeout و خطا

`Client` برای شبکه `Timeout` دارد و خطاهای `HTTP`، `Socket Timeout` و `Parsing JSON` را به `Exception` اختصاصی تبدیل می‌کند. این کار باعث می‌شود `Repository` و `UI` بتوانند خطا را به شکل کنترل‌شده مدیریت کنند.

۴.۱۰ Fallback

`Fallback` یکی از مهم‌ترین اصول پایداری پروژه است. در `Lookup ID` ابتدا `IMDb Metadata` امتحان می‌شود، سپس `OMDb` و در نهایت یک نتیجه حداقلی شامل `IMDb ID` تولید می‌شود. در `Episode` نیز `IMDb` و سپس `OMDb` امتحان می‌شوند.

۵.۱۰ امنیت و اطلاعات حساس

در Backend مقدار IMDb Session از Cookie استخراج می‌شود و در های Header بعدی استفاده می‌شود؛ بنابراین چنین اطلاعاتی نباید در Log عمومی چاپ شوند. کد فعلی بخشی از Session ID را در Log کوتاه می‌کند، اما در محیط Production باید سیاست Logging دقیق و محدود باشد. برای OMDb، کلید API در Flutter از `String.fromEnvironment('OMDB_API_KEY')` دریافت می‌شود و به صورت ثابت در فایل Dart قرار نگرفته است. این روش بهتر از Hard-code کردن کلید است، هر چند در یک اپلیکیشن Client-side همچنان باید در نظر داشت که Secret واقعی را نمی‌توان به صورت مطلق از کاربر نهایی پنهان کرد.

۶.۱۰ ریسک وابستگی به API داخلی IMDb

بزرگ‌ترین محدودیت فنی این معماری وابستگی به های Endpoint داخلی و Query Persisted. هاست Hash این ها Endpoint قرارداد عمومی ثابت پروژه نیستند. تغییر در Header، Hash، Name، Operation ساختار JSON یا سیستم Session می‌تواند باعث شکست ارتباط شود.

۷.۱۰ وابستگی به Playwright

در Backend، نصب Chromium و هماهنگی نسخه Playwright ضروری است. مصرف RAM و زمان Startup نیز بیشتر از یک Client HTTP ساده است. بنابراین Playwright باید در جایی استفاده شود که واقعاً Browser Context لازم است و نباید برای هر درخواست API به صورت بی‌قید و شرط Browser جدید باز شود.

۸.۱۰ تست‌های انجام‌شده

در پروژه برای بخش‌های مختلف تست‌هایی وجود دارد که های Endpoint، Details، Trending، Suggestions و قابلیت‌های Watch و Review را بررسی می‌کنند. علاوه بر تست خود کد، آزمایش‌های عملی اتصال نیز بخشی از فرایند توسعه بودند: requests و urllib در سناریوی اولیه موفق نبودند و Playwright در همان محیط توانست صفحه IMDb را بارگذاری کند.

۹.۱۰ تفاوت مشاهده تجربی و نتیجه قطعی

مهم است که در گزارش علمی بین «مشاهده آزمایشگاهی» و «علت قطعی» تفاوت گذاشته شود. مشاهده ما این بود که requests و urllib در آزمایش انجام‌شده نتیجه مورد نیاز را ندادند و Playwright موفق شد. از روی کد می‌توان نتیجه گرفت که Cookie Session و اجرای JavaScript برای راه‌حل انتخاب‌شده اهمیت داشته‌اند؛ اما بدون بررسی دقیق پاسخ‌های خام شبکه نمی‌توان ادعا کرد که فقط یک مکانیزم خاص عامل شکست بوده است.

۱۰.۱۰ بهبودهای پیشنهادی

برای یک نسخه Production می‌توان چند بهبود در نظر گرفت:

۱. قراردادن همه، ها Endpoint ها Operation ها و Hash در یک لایه مرکزی قابل به‌روزرسانی.
۲. افزودن Retry با Backoff برای خطاهای موقتی.
۳. استفاده از Pool Browser به جای ساخت Chromium جدید در هر Scrape.
۴. ثبت Metrics برای نرخ موفقیت API و زمان پاسخ.
۵. محدود کردن های Log حساس و حذف کامل ID Session از Log در Production.
۶. در صورت امکان، استفاده از یک API رسمی و پایدار به جای های Endpoint داخلی.
۷. جداسازی کامل Proxy و Scraper و مشخص کردن اینکه هر قابلیت از کدام مسیر استفاده می‌کند.

۱۱.۱۰ جمع‌بندی نهایی

راه‌حل نهایی پروژه نتیجه چند مرحله آزمون و خطا بوده است. اتصال ساده HTTP برای سناریوی آزمایش شده کافی نبود، Playwright توانست Context Browser مناسب ایجاد کند، Session از Cookie استخراج شد و سپس درخواست‌های API با های Header لازم ادامه پیدا کردند. در Flutter Client نیز ارتباط مستقیم با IMDb با dart:io/HttpClient و های GraphQL Query پیاده‌سازی شده است. این معماری در کنار OMDb و های Fallback باعث شده Client بتواند در برابر شکست یک منبع، در برخی مسیرها همچنان داده قابل استفاده تولید کند؛ با این حال وابستگی به های Endpoint داخلی IMDb مهم‌ترین محدودیت راه‌حل باقی می‌ماند.

پیوست آ

پیوست: مرجع ها Endpoint و های Operation

پروژه

آ.۱ جدول های Endpoint اصلی

جدول آ.۱: ها Endpoint و های Operation مورد استفاده

نام	روش	Operation / مسیر	کاربرد
Suggestion	GET	v3.sg.media-imdb.com/suggestion/x/...json	پیشنهاد های جست و جو
GraphQL	GET/POST	tracing.graphql.imdb.com/GraphQL	مشترک Endpoint
Trending	GET	Trending	آثار ترند
Search Advanced	POST/GET	AdvancedTitleSearch	جست و جوی پیشرفته
Metadata	GET	FavoriteTitlesMetadata	اطلاعات عنوان بر اساس tconst
Overview Series	GET	HERO_SUB_NAV_EPISODE	نکمیلی سریال
Episodes Season	GET	EpisodeRatings_SeasonDetail	یک فصل
OMDb	GET	www.omdbapi.com	منبع جانبی و Fallback

آ.۲ های Query Persisted Hash در Client

جدول ۲.آ: های Hash موجود در کد Client

Hash ۲۵۶SHA-	Operation
4fc66817a78c3046e0cedef747033d5ac2711080338a59a366630f9742c1	Trending
32519bc74ceb6be628fe452c0e59a48bcf8ca91fc550dd5de43ab200acd52	AdvancedTitleSearch
f6473ec76e947098d2585c35f0891c4b47c2ef261c14b7c82902ae196d1b	FavoriteTitlesMetadata
a4c9c2cca81733ebabbf5e317e3da7f2a4a02069d406bec001ed611c80e4	HERO_SUB_NAV_EPISODE
a7aa5ba917bd6e519570375e6f3f570ad3503e6a9d202b1fa4cb5ae6a56d	EpisodeRatings_SeasonDetail

۳.آ مسیرهای Gateway محلی

جدول ۳.آ: های Gateway Django Endpoint

مسیر	کاربرد
/api/search/	Suggestion Search
/api/episodes/	دریافت های Episode فصل
/api/video/<video_id>/	اطلاعات استریم ویدئو
/api/suggest/name/	پیشنهاد نام
/api/search/advanced/	Search Advanced
/api/discover/movies/popular/	فیلم های محبوب
/api/discover/tv/popular/	سریال های محبوب
/api/discover/new/	آثار جدید
/api/discover/top-rated/	آثار با امتیاز بالا
/api/discover/recommended/	پیشنهاد بر اساس ژانر

۴.آ نکته پایانی

تمام نام ها، ها Operation ها Hash و مسیرهای این پیوست از کد پروژه استخراج شده اند. چون بخش مهمی از این ها API داخلی هستند، نباید آن ها را قرارداد رسمی و دائمی IMDb فرض کرد. این جدول در درجه اول یک Snapshot فنی از نسخه ای است که در پروژه پیاده سازی و آزمایش شده است.